МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: «Поиск с возвратом»

Студентка гр. 4345 _____ Бодарева С.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы.

Изучить и реализовать алгоритм поиска с возвратом. Провести исследование его работы на различных размерах входных данных.

Задание.

У Вовы много квадратных обрезов доски. Их стороны (размер) изменяются от 1 до $N - 1$, и у него есть неограниченное количество обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N. Он может получить её, собрав из уже имеющихся обрезков (квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков (см. Рис. 1).

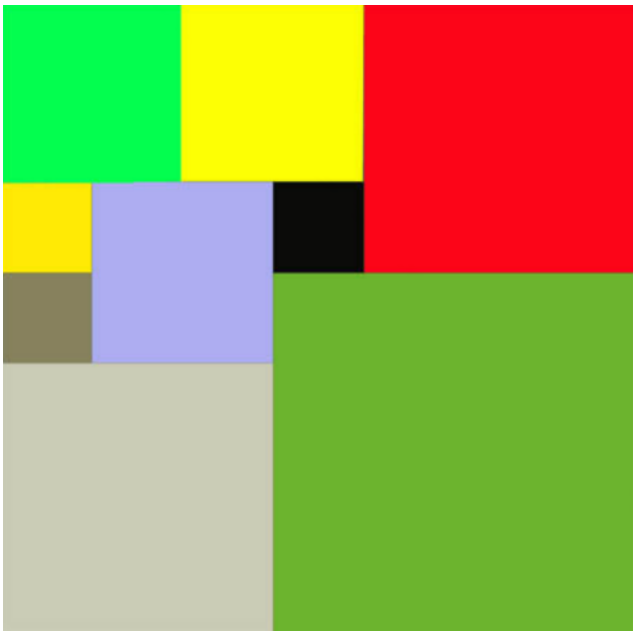

Рисунок 1. Столешница 7×7.

Внутри столешницы не должно быть пустот, обрезки не должны выходит за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N $(2 \leq N \leq 40)$.

Выходные данные

Одно число K, задающее минимальное количество обрезков (квадратов), из которых можно построить столешницу (квадрат) заданного размера N.

Далее должны K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 3р.

Рекурсивный бэктрекинг. Исследование кол-ва операций от размера квадрата.

Основные теоретические положения.

Поиск с возвратом (бэктрекинг) – общий метод нахождения решений задачи, в которой требуется полный перебор всех возможных вариантов в некотором множестве М.

Решение задачи методом поиска с возвратом сводится к последовательному расширению частичного решения. Если на очередном шаге такое расширение провести не удается, то возвращаются к более короткому частичному решению и продолжают поиск дальше. Данный алгоритм позволяет найти все решения поставленной задачи, если они существуют. Для ускорения метода стараются вычисления организовать таким

образом, чтобы как можно раньше выявлять заведомо неподходящие варианты. Зачастую это позволяет значительно уменьшить время нахождения решения.

Данный подход широко применяется в области разработки программного обеспечения, особенно в решении задач комбинаторной оптимизации, где требуется найти наилучшее решение из множества возможных комбинаций.

Выполнение.

Рассмотрим задачу заполнения квадрата со стороной N квадратами со сторонами в диапазоне от 1 до $N - 1$. Её можно разбить на два случая:

1) N – чётное число. В таком случае квадрат всегда можно разбить на 4 равных квадрата со сторонами $\frac{N}{2}$ (см. Рис. 2).

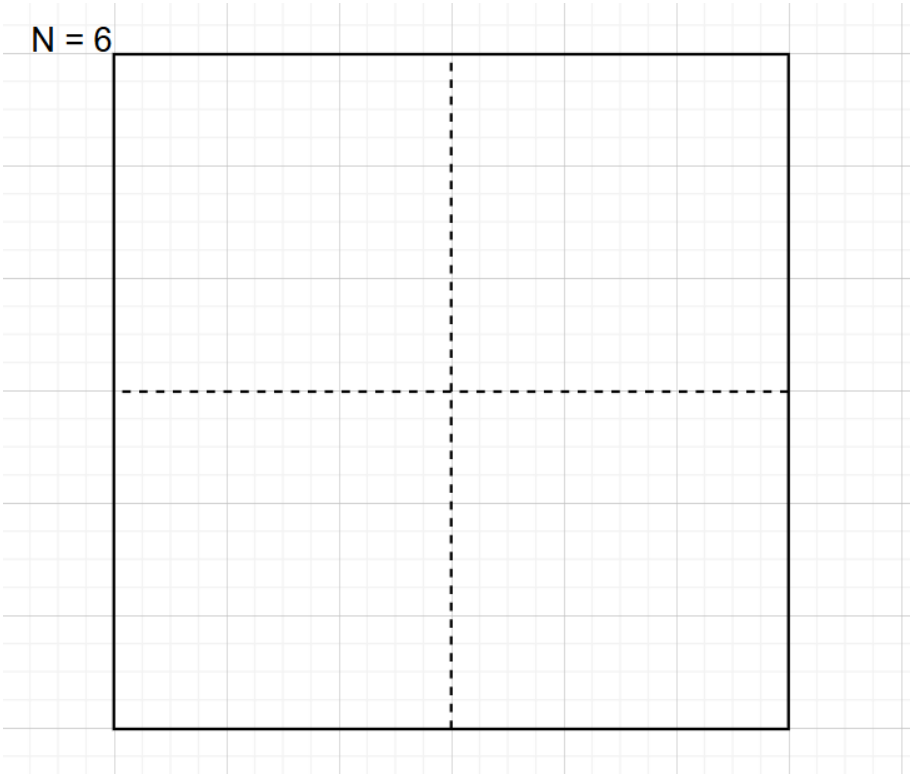

Рисунок 2. Пример разбиения для квадрата с чётного N.

2) N – нечётное число. Этот случай разбивается ещё на два случая для простого и составного N.

Если N – простое число, то необходимо размесить в левом верхнем углу квадрат со стороной $\frac{N+1}{2}$, а в левом нижнем и правом верхнем квадраты со стороной $\frac{N-1}{2}$. Тогда в правом нижнем углу останется фигура, которая будет представлять из себя квадрат со стороной $\frac{N+1}{2}$ с вырезанным кусочком (см. Рис. 3).

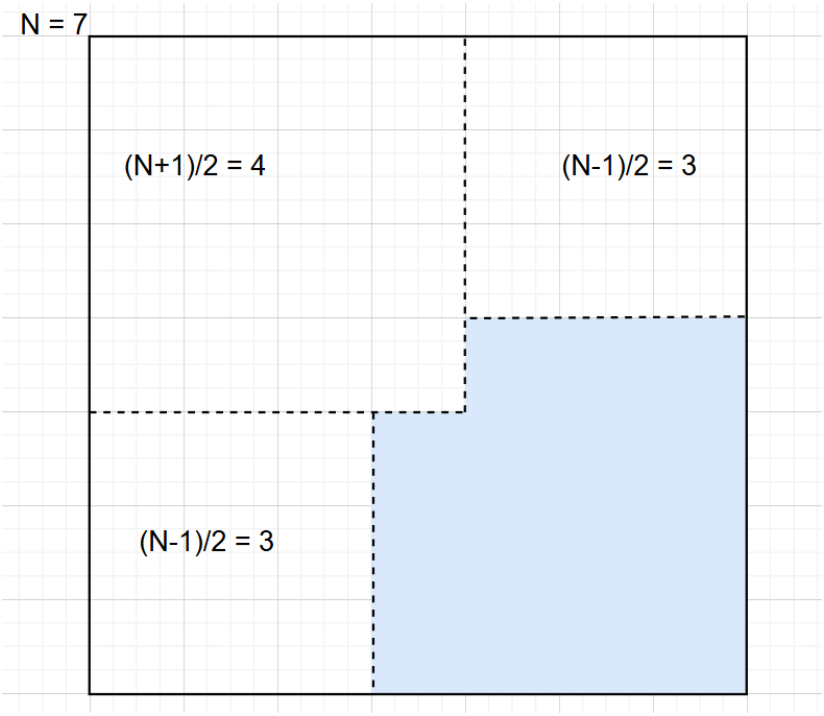

Рисунок 3. Пример вставки трёх квадратов для простого N.

После расстановки трёх квадратов, закрашенная часть будет заполнена при помощи поиска с возвратом. Для каждой клетки заполняемой будет вычислен максимальный размер квадрата, который можно разместить в ней так, чтобы рассматриваемая клетка оказалась в левом верхнем углу. Затем в текущую клетку поочередно вставляются все возможные квадраты, и алгоритм запускается снова для оставшейся фигуры. При этом, если количество квадратов в текущем разбиении превысило количество квадратов в уже найденном, то дальнейшее разбиение не происходит, а текущий случай отбрасывается. Когда разбиение покрывает всю заштрихованную область, оно сравнивается с лучшим, если лучшего нет, то становится им.

Если N – нечётное составное число, то задачу можно свести к случаю с простым числом. Для этого необходимо представить размер столешницы в виде $N = D * K$, где D – минимальный делитель N, отличный от 1. Тогда можно будет решить более простую задачу для квадрата со стороной D, итоговое разбиение которого будет совпадать с разбиением для N, только получившиеся в D квадраты нужно будет увеличить в K раз (см. Рис. 4). Для координаты a новая координата будет рассчитана как $(a - 1) * K + 1$ (прибавляем единицу, так как считаем координаты с 1, а не с 0).

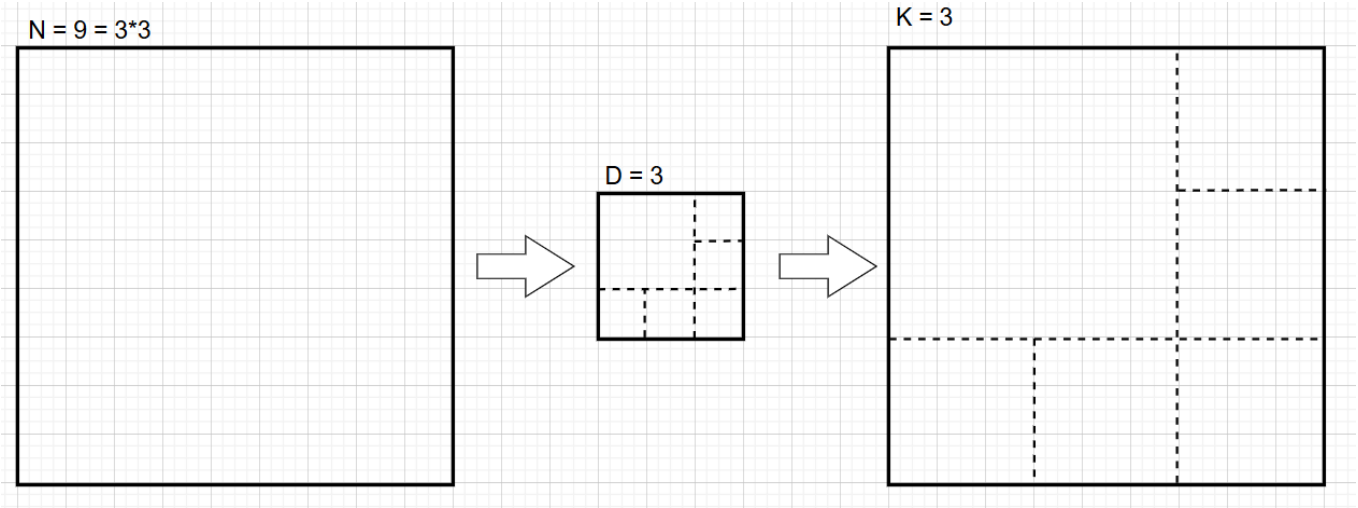

Рисунок 4. Пример применения масштабирования для нечётного составного N.

Для реализации данного алгоритма были созданы:

1) *class Piece* – класс обрезка. В нем содержатся поля *column* и *row*, отвечающие за координату левого верхнего угла квадратного обрезка, а также поле *pieceSize*, отвечающее за размер стороны квадрата. Для данного класса перегружен оператор вывода в поток.

2) *class Table* – класс стола. В нем содержатся поле *tableSize*, хранящее размер стола, кроме того, существуют поля *size* и *scale*. *Size* – минимальный простой делитель стороны стола, *scale* – частное от деления стороны стола на *size*. Для хранения лучшей и текущей расстановок есть поля *bestSplitting* и *currentSplitting* – вектора объектов класса *Piece*. В классе *Table* реализованы следующие методы:

- *int findMinDivisor(int n)* – поиск минимального простого делителя числа *n*. Если такого делителя не нашлось, то будет возвращено само число *n*;
- *bool isUsed(int column, int row)* – проверка на то, содержится ли клетка с координатами *(column, row)* в каком-либо квадрате текущей расстановки;
- *void backtracking(int column, int row, int filledPart)* – данный метод реализует рекурсивный алгоритм поиска с возвратом, описанный ранее, на вход подаются координаты клетки, в которой начинается бэктрекинг и площадь заполненной части;
- *void scaling()* – метод, который масштабирует лучшую расстановку в случае, если *N* нечётное;
- *void findBestSplitting()* – в данном методе определяется ход решения задачи. Для чётных чисел лучшее разбиение определяется сразу, для нечётных запускается алгоритм поиска с возвратом.

Для класса *Table* перегружен оператор вывода в поток.

Проведено исследование изменения количества итераций поиска решения выполнения в зависимости от размера квадрата.

Результаты исследования см. в Таблице 1.

Таблица 1. Количество итераций в зависимости от размера квадрата.

Размер квадрата	Количество итераций	Размер квадрата	Количество итераций
3	3	23	20112
5	9	25	9
7	24	27	3
9	3	29	136384
11	204	31	297125
13	424	33	3
15	3	35	9
17	2199	37	1560203
19	5446	39	3
21	3		

График зависимости количества итераций от размера квадрата см. на Рис.5.

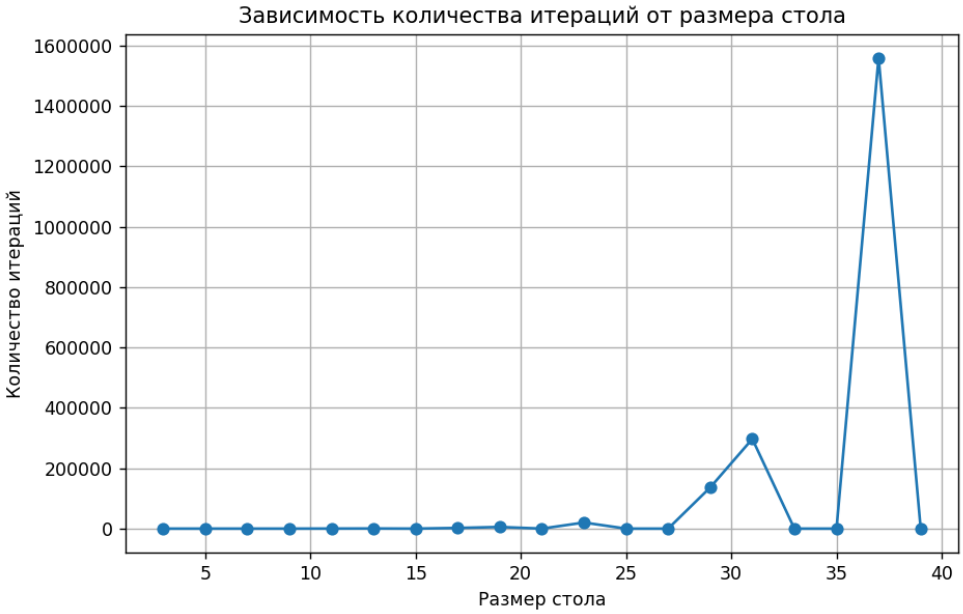

Рисунок 5. График зависимости количества итераций от размера квадрата.

Для чётных чисел алгоритм поиска с возвратом не вызывается, количество итераций равно 0, поэтому они не включены в диаграмму. Использование масштабирования значительно сокращает количество итераций для нечётных составных чисел.

Изучим изменение количества операций на размерах квадратов, которые являются простыми числами. Результаты см. на Рис. 6.

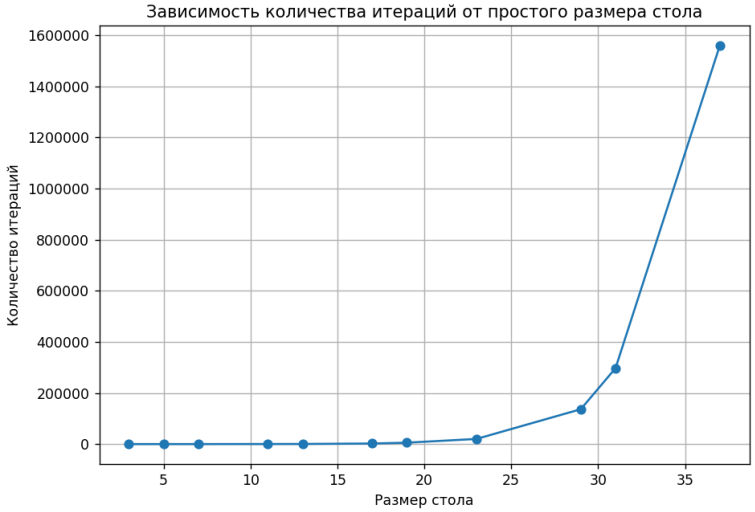

Рисунок 6. График зависимости количества итераций от размера квадрата, являющегося простым числом.

Для простых чисел сложность алгоритма экспоненциальная.

Проведено исследование времени поиска решения выполнения в зависимости от размера квадрата.

Результаты исследования см. в Таблице 2.

Размер квадрата	Время, с	Размер квадрата	Время, с
3	0.000009	23	0.0064
5	0.000011	25	0.000011
7	0.000018	27	0.000009
9	0.000009	29	0.0484
11	0.000082	31	0.0905
13	0.000118	33	0.000009
15	0.000009	35	0.00005
17	0.000655	37	0.4424
19	0.001837	39	0.000009
21	0.000009		

График зависимости времени разбиения от размера квадрата см. на Рис. 7.

Рисунок 7 – Зависимость времени выполнения от размера квадрата

Сложность алгоритма по времени $O\left(c^{N/4}\right), c > 1$ – экспоненциальная

Сложность алгоритма по памяти: $O(1)$ для чётных N, $O(N^2)$ для нечётных N – квадратичная

Исходный код программы см. в Приложении А.

Результаты тестирования см. в таблице 2.

Таблица 2. Результаты тестирования

№	Входные данные	Выходные данные	Комментарии
1	7	9 1 1 4 5 1 3 1 5 3 5 4 2 7 4 1 4 5 1 7 5 1 4 6 2 6 6 2	Пример из задания.
2	27	6 1 1 18 19 1 9 1 19 9 19 10 9 10 19 9 19 19 9	Составное нечётное число, можно масштабированием свести к задаче с N=3.
3	29	14 1 1 15 16 1 14 1 16 14 16 15 2 18 15 5 23 15 7 15 16 1 15 17 3 15 20 3 18 20 3 21 20 2 21 22 1 22 22 8 15 23 7	Простое число.
4	10	4 1 1 5 6 1 5 1 6 5 6 6 5	Чётное число, лучшее разбиение определяется без перебора.

Вывод.

Был изучен и реализован алгоритм поиска с возвратом на примере задачи о заполнении столешницы квадратными обрезками. Было проведено исследование зависимости количества вызовов функции перебора в зависимости от размера столешницы. Исследование показало, что для чётных чисел задача тривиальная и не требует перебора вариантов, поскольку любой квадрат с чётной стороной всегда можно разбить на четыре со сторонами, равными половине длины стороны исходного квадрата. Для простых чисел необходимо использовать перебор. Нетрудно заметить, что с ростом числа количество итераций, необходимых для нахождения лучшего решения, возрастает экспоненциально, несмотря на то что неоптимальные решения отбрасываются. Для нечётных составных чисел задача сводится к поиску решения для минимального простого делителя, что уменьшает количество необходимых итераций. Таким образом, лучший случай для данного алгоритма – чётное число, а худший – простое.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: lb1.cpp

```
#include <iostream>
#include <vector>

class Piece{
public:
    int column;
    int row;
    int pieceSize;
    Piece(int  column,  int  row,  int  pieceSize):column(column),  row(row),
pieceSize(pieceSize){}

    friend std::ostream& operator << (std::ostream& out, const Piece& piece){
        return out << piece.column << " " << piece.row << " " << piece.pieceSize;
    }
};


class Table{
    int tableSize;
    int size;
    int scale;
    std::vector<Piece> bestSplitting;
    std::vector<Piece> currentSplitting;

    int findMinDivisor(int n){
        for(int i{2}; i< n; ++i){
            if(n % i == 0){
                return i;
            }
        }
        return n;
    }

    bool isUsed(int column, int row){
        for(const Piece& piece : currentSplitting){
            if((row >= piece.row) && (row < piece.row + piece.pieceSize) &&
(column >= piece.column) && (column < piece.column + piece.pieceSize))
                return true;
        }
        return false;
    }

    void backtracking(int column, int row, int filledPart) {
        for (int y{row}; y <= size; ++y) {
            for (int x{column}; x <= size; ++x) {
                if (!isUsed(x, y)) {
                    int squareSize = std::min(size - x,  size - y) + 1;
                    for (const Piece& piece : currentSplitting) {
                        if (piece.row + piece.pieceSize > y && piece.column >
x)
                            squareSize  =  std::min(squareSize,  piece.column  -
x);
                    }
                    for (int s{squareSize}; s > 0; --s) {
                        currentSplitting.push_back(Piece(x, y, s));
                        if (filledPart + s * s == size * size) {
```

```
                            if (currentSplitting.size() < bestSplitting.size()
|| bestSplitting.size() == 0) {
                                bestSplitting = currentSplitting;
                            }
                        }
                        else {
                            if     (currentSplitting.size()     +     1     <
bestSplitting.size() || bestSplitting.size() == 0) {
                                backtracking(x + s, y, filledPart + s * s);
                            }
                        }
                        currentSplitting.pop_back();
                    }
                    return;
                }
            }
            column = (size + 1) / 2;
        }
    }

    void scaling(){
        for (Piece& piece : bestSplitting){
            piece.column = (piece.column-1) * scale + 1;
            piece.row = (piece.row-1) * scale + 1;
            piece.pieceSize = piece.pieceSize * scale;
        }
    }

public:
    Table(int                                tableSize):tableSize(tableSize),
size(findMinDivisor(tableSize)), scale(tableSize/size){}

    friend std::ostream& operator << (std::ostream& out, const Table& table){
        out << table.bestSplitting.size() << "\n";
        for(int i{0}; i < table.bestSplitting.size(); ++i){
            out << table.bestSplitting[i] << "\n";
        }
        return out;
    }

    void findBestSplitting(){
        if(tableSize % 2 == 0){
            bestSplitting.push_back(Piece(1, 1, tableSize/2));
            bestSplitting.push_back(Piece(tableSize/2+1, 1, tableSize/2));
            bestSplitting.push_back(Piece(1, tableSize/2+1, tableSize/2));
            bestSplitting.push_back(Piece(tableSize/2+1,        tableSize/2+1,
tableSize/2));
        }
        else{
            currentSplitting.push_back(Piece(1, 1, (size+1)/2));
            currentSplitting.push_back(Piece((size+1)/2+1, 1, (size-1)/2));
            currentSplitting.push_back(Piece(1, (size+1)/2+1, (size-1)/2));
            backtracking((size+1)/2, (size+1)/2, 0.25*(3*size*size-2*size+3));
            scaling();
        }
    }

};


int main(){
    int n;
    std::cin >> n;
    Table t(n);
```

```
    t.findBestSplitting();
    std::cout << t;
    return 0;
}
```